

Fundamentos de Programación

Tema 7: Introducción a la programación funcional

Contenidos

1.- Introducción	2
2.- Expresiones lambda	4
3.- Funciones de orden superior	11
4.- Streams	18
5.- Referencias a métodos	29

Tema

7

INTRODUCCIÓN A LA PROGRAMACIÓN FUNCIONAL

Hasta ahora todo el curso ha consistido en el estudio de la programación estructurada y la programación orientada a objetos, que han sido los modelos de programación dominantes en las últimas décadas. Sin embargo, hoy día los principales lenguajes están incorporando elementos de otro modelo de programación completamente diferente: la programación funcional

1.- Introducción

Cuando surgieron los lenguajes de programación de alto nivel, pronto comenzaron a aparecer multitud de lenguajes diferentes, y con ellos, “ideas” e “innovaciones” con las que un lenguaje pretendía diferenciarse de los demás. Algunos lenguajes eran prácticamente una copia de otros, pero de vez en cuando aparecían lenguajes que pretendían enfocar la programación desde otro punto de vista, cambiando así la “forma de programar”. En esos lenguajes aparecían cosas nunca vistas, que cambiaban la forma de plantear los problemas y hacer los programas. Surgieron así los **paradigmas de la programación**, que son categorías en las que clasificamos los lenguajes, según los elementos que encontramos en ellos.

- Paradigma estructurado: Apareció con los primeros lenguajes de programación (Fortran, Algol, C, etc) y consiste en programar algoritmos expresando paso a paso cómo obtener la solución mediante bucles, condicionales, funciones¹, etc
- Paradigma orientado a objetos: Apareció en los años 70 (Simula 67, Smalltalk, C++, etc) y consiste en programar utilizando clases y objetos, como ya hemos visto. Sus conceptos fundamentales son el encapsulamiento, la herencia y el polimorfismo.

De forma paralela, aunque no tan predominantes como los paradigmas estructurado y orientado a objetos, aparecieron lenguajes que usaban otros paradigmas. Uno de ellos era el paradigma funcional, que tiene su origen en una rama de las matemáticas llamada **cálculo lambda**, y que consiste en la descripción de los programas usando funciones matemáticas.

En los lenguajes puramente funcionales (como Haskell o Erlang) no existen bucles ni clases, y la funcionalidad de los programas se describe mediante funciones que nos recuerdan en su forma a las funciones matemáticas $y = f(x)$. Los lenguajes funcionales tienen una sintaxis que no se parece en nada a la de los lenguajes que hemos estudiado. Algunos de sus términos (funciones puras, inmutabilidad, recursividad) los podemos encontrar en los lenguajes convencionales, pero otros son completamente diferentes (lambdas, tipos algebraicos de datos, funtores, mónadas, comprensiones monádicas, coproducto de mónadas...)

¹ El concepto de **función** está muy relacionado con el de método, pero si somos muy estrictos, no son lo mismo. El lenguaje Java **no tiene funciones**. Otros lenguajes como Kotlin, JavaScript o Python, tienen tanto **métodos** como **funciones**.

```

tryCapitalizeM :: (Functor m, Monad m, Walkable Inline a, Default a, Eq a) =>
  (String -> m a) -> String -> Bool -> m a
tryCapitalizeM f varname capitalize
  | capitalize = do
    res <- f (capitalizeFirst varname)
    case res of
      xs | xs == def -> f varname >= walkM capStrFst
        | otherwise -> return xs
  | otherwise = f varname
where
  capStrFst (Str s) = return $ Str $ capitalizeFirst s
  capStrFst x = return x

```

Fig 1 – Código fuente escrito en lenguaje Haskell

En los últimos años se ha comprobado que los lenguajes puramente funcionales funcionan muy bien en situaciones donde los lenguajes clásicos han presentado problemas, como por ejemplo los programas multitarea, cuando varios hilos de ejecución comparten recursos y deben funcionar al mismo tiempo. También se ha visto que son menos propensos a fallos (es más fácil describir un programa, que programar una solución) y se corrigen más fácilmente los errores.

Por estos motivos, los lenguajes de programación habituales (Java, Python, e incluso C++) están incorporando elementos propios de los lenguajes funcionales. Sin embargo, este tipo de lenguajes no son 100% funcionales y no poseen todas las características de la programación funcional. Digamos que son lenguajes imperativos y orientados a objetos en los que se puede hacer un poco de programación funcional. También existen lenguajes híbridos como Scala, que son completamente funcionales, pero pueden ser usados como imperativos y orientados a objetos si se desea (por ese motivo, los puristas dicen que no son 100% funcionales, ya que es muy fácil “salirse” de los elementos funcionales para usar los otros).

En este tema nos centraremos en ver cómo Java² ha incorporado elementos básicos de programación funcional. De hecho, pese a su sencillez, la introducción de estos elementos es una de las contribuciones más importantes a Java en toda su historia y una de las características que más se le valoran. Todas las librerías y frameworks modernos de Java la usan de una u otra forma, y es el estilo de programación recomendado en Java.



Fig 2 – Elementos de la programación funcional en Java

² La programación funcional se introduce en Java en 2014, en la versión Java 8

2.- Expresiones lambda

Hasta ahora hemos usado variables para guardar los datos que maneja nuestro programa. Por ejemplo, si tenemos una persona, su sueldo puede ser un número con decimales (double), y su fecha de nacimiento un objeto LocalDate. Estamos muy acostumbrados a crear una variable por cada pieza de información con la que hay que trabajar.

Sin embargo, también existe la posibilidad de hacer una variable que guarde un trozo de código fuente. Por ejemplo, podemos hacer una variable que guarde en su interior el código fuente que abra un archivo y muestre su contenido en la pantalla. Dicho código estará guardado en la variable, y cuando queramos lanzarlo, llamaremos a un método de la variable (llamado “método activador”) que lo pondrá en funcionamiento.

En la programación funcional de Java se usan variables que guardan código fuente. La variable posee un método activador que ejecuta el código fuente que guarda en su interior

El código fuente que se guarda en la variable también puede recibir parámetros y devolver resultados. Por ejemplo, podemos hacer una variable que guarde código fuente que recibe dos números como parámetros y nos devuelve su suma. De esta forma, cuando queramos lanzar dicho código, el método activador de la variable recibirá los dos números que queremos sumar y después, como en cualquier otro método, guardaremos el resultado de la llamada al método activador (que en este ejemplo, devolvería la suma) en otra variable.

Tras leer las explicaciones anteriores, surgen dos preguntas inmediatamente:

- ¿Qué tipo de dato tienen las variables que guardan código fuente?
- ¿Cómo se rellena una variable con código fuente?

Vamos a responder estas preguntas en los siguientes apartados:

2.1.- Interfaces funcionales predefinidas

Los tipos de datos de las variables que guardan código fuente se denominan **interfaces funcionales** y hay varios tipos posibles, según los parámetros que recibe el código fuente que se quiere guardar en la variable, y el resultado que produzca.

Tipo de dato	Explicación	Representación	Método activador
Runnable	Código que no recibe parámetros y ejecuta una acción sin devolver ningún resultado	() -> ()	+ void run()
Function<A,B>	Código que recibe un dato de tipo A y nos devuelve un dato de tipo B	A -> B	+ B apply(A dato)
Predicate<A>	Código que recibe un dato de tipo A y nos devuelve un boolean	A -> Boolean	+ boolean test(A dato)
Consumer<A>	Código que recibe un dato de tipo A, lo procesa y no devuelve ningún resultado	A -> ()	+ void accept(A dato)
Supplier<A>	Código que no recibe parámetros y nos genera un objeto de tipo A	() -> A	+ A get()

Mirando esa tabla, podemos saber cuál es el tipo de dato de la variable que vamos a necesitar, según sea el código fuente que querremos guardar en ella. Por ejemplo:

- Si queremos guardar en una variable el código fuente que abre un archivo llamado factura.txt y mostrarlo en pantalla, usaremos el tipo **Runnable**, ya que dicho código se limita a abrir el archivo, leerlo y mostrarlo en pantalla, sin devolver nada
- Si queremos guardar en una variable el código fuente que recibe un número entero y nos devuelve su raíz cuadrada, usaremos el tipo **Function<Integer,Double>**, ya que como parámetro se recibe un número entero y el código guardado en la variable devolverá un double con el valor de su raíz cuadrada.
- Si queremos guardar en una variable el código fuente que recibe como parámetro una cadena de caracteres y nos devuelve su número de espacios, usaremos el tipo **Function<String,Integer>**, ya que el parámetro que recibe el código es una cadena de caracteres, y se devolverá un entero con su número de espacios.
- Si queremos guardar en una variable el código fuente que recibe un objeto Persona y nos devuelve true si es mayor de edad, usaremos el tipo **Predicate<Persona>**, ya que se recibe como parámetro un objeto Persona y el código guardado en la variable nos devuelve un boolean.
- Si queremos guardar en una variable el código fuente que recibe un String y lo muestra en pantalla, usaremos el tipo **Consumer<String>**, porque se recibe como parámetro un String y se realiza una acción sobre él que no produce ningún resultado.
- Si queremos guardar en una variable el código fuente que nos genera un número aleatorio entre 10 y 20, usaremos el tipo **Supplier<Integer>**, ya que sin ningún parámetro de entrada, se genera un número entero.

Los tipos anteriores se denominan **interfaces funcionales predefinidos** porque vienen ya programados en Java en el paquete **java.util.function**. De hecho, hay muchos más tipos³ e incluso podemos crear nuestros propios tipos de interfaces funcionales.

De todos los tipos de interfaces funcionales predefinidos, destacan los 5 que hemos puesto en la tabla anterior (que son los más importantes) y sus versiones para recibir dos parámetros de entrada, que comienzan por **Bi**. Por ejemplo:

- Si queremos guardar en una variable el código que recibe dos números y nos devuelve su división con decimales, usaremos el tipo **BiFunction<Integer,Integer,Double>** porque recibe dos números como parámetro (los dos primeros Integer) y nos devuelve el resultado de dividirlos con decimales.
- Si queremos guardar el código fuente que recibe dos String y nos devuelve true si uno está contenido en el otro, usaremos el tipo **BiPredicate<String,String>**, ya que se recibe dos String y nos devuelve un boolean.
- Si queremos guardar el código que recibe dos palabras y las muestra en pantalla una tras otra, usaremos el tipo **BiConsumer<String,String>**, ya que se hace una acción sobre las palabras, sin devolver nada.

³ Pueden consultarse en: <https://docs.oracle.com/javase/8/docs/api/java/util/function/package-summary.html>

2.2.- Expresiones lambda



Ahora que ya conocemos los tipos de datos que nos van a permitir guardar código fuente en una variable, vamos a ver cómo podemos rellenar esas variables con código.

Para rellenar una variable con código fuente se usa una notación especial llamada **expresión lambda**, que consiste en poner la lista de parámetros entre paréntesis, una flecha -> y después, entre llaves, el código fuente que se quiere guardar en la variable.

Por ejemplo, para crear una variable llamada **código** que contenga el código fuente que abre un archivo llamado factura.txt y nos muestra en pantalla su contenido haríamos esto:

```
1. Runnable codigo = () -> {
2.     try{
3.         FileReader a = new FileReader("factura.txt");
4.         BufferedReader b = new BufferedReader(a);
5.         String linea="";
6.         while(linea!=null){
7.             linea = b.readLine();
8.             System.out.println(linea);
9.         }
10.        b.close();
11.        a.close();
12.    }catch(IOException e){
13.        System.out.println("Error al leer el archivo");
14.    }
15. };
```

La forma de escritura que estamos viendo para definir el código fuente que se guarda en la variable se llama **expresión lambda**, y se reconoce fácilmente porque aparece la flecha -> seguida de unas llaves que encierran el código que se guarda en la variable.

La línea anterior está creando una variable llamada **código** y la está rellenando con código fuente que abre factura.txt y muestra todas sus líneas en la pantalla. Dicho código fuente no se ejecuta hasta que no llamemos al **método activador** de la variable **código**, que en el caso del tipo Runnable, se llama **run** (ver la tabla del apartado anterior). O sea, si tras crear la variable Runnable queremos ejecutar el código fuente que contiene, llamaríamos a su método run así:

```
1. // ejecuta el código fuente que contiene la variable de tipo Runnable
2. codigo.run()
```

Vamos a ver otro ejemplo. Queremos guardar en una variable llamada **codigo2** el código fuente que recibe un número y nos devuelve su raíz cuadrada. Lo haríamos así⁴:

```
1. Function<Integer,Double> codigo2 = (Integer n) -> {
2.     return Math.sqrt(n)
3. };
```

Para activar el código fuente y calcular la raíz cuadrada de un 8, llamaríamos al método activador de la variable **codigo2**, que en este caso se llama **apply**. Pasaríamos a dicho método el valor 8, que es el número cuya raíz queremos calcular.

⁴ Las interfaces funcionales que usaremos en este tema solo trabajan con objetos. También existen interfaces funcionales que trabajan con tipos básicos, pero no las vamos a ver aquí.

```
1. double raiz = codigo2.apply(8);
2. System.out.println("La raiz de 8 es : "+raiz);
```

Veamos un último ejemplo. Queremos guardar en una variable el código fuente que recibe una frase y nos devuelve true si su número de caracteres es par. Escribiremos esto:

```
1. Predicate<String> codigo3 = (String frase) -> {
2.     return frase.length() % 2 == 0;
3. };
```

Si queremos activar el código fuente que hay guardado en la variable **codigo3** para que nos diga si la frase “hola amigos” tiene un número par de caracteres, llamaremos a su método activador, que en el caso del Predicate se llama **test**

```
1. boolean longitudPar = codigo3.test("hola amigos");
2. System.out.println(longitudPar? "longitud par" : "longitud impar");
```

Ejercicio 1: Usa expresiones lambda para crear las variables adecuadas que guarden los siguientes fragmentos de código fuente:

- a) No se reciben parámetros y se muestra en pantalla la frase “hola mundo”
- b) Como parámetro se recibe un número entero y nos devuelve su cuadrado
- c) No se reciben parámetros y nos devuelve la fecha actual
- d) Como parámetro se recibe un List<Integer> y nos dice si está vacío
- e) Como parámetro recibe la ruta de un archivo y si no existe, lo crea.
- f) Como parámetro se recibe una frase y devuelve el número de espacios que hay en ella
- g) Como parámetro se reciben dos números enteros y nos devuelve el máximo de ellos.
- h) Como parámetro se recibe un número entero con un dni y nos devuelve la letra correspondiente a ese dni
- i) Como parámetro recibe una edad y nos devuelve true si es mayor de edad
- j) Como parámetro se recibe la ruta de un archivo y devuelve su tamaño en bytes
- k) Como parámetro recibe una LocalDate y nos devuelve una cadena de caracteres que lo expresa en formato día/mes/año
- l) Como parámetro se recibe un número y nos devuelve true si el número es primo
- m) No se reciben parámetros, y muestra en pantalla la hora actual
- n) Como parámetro recibe la ruta de un archivo y muestra todas sus líneas en pantalla
- o) Recibe un número entero y hace una pausa de esa cantidad de segundos
- p) Como parámetro se recibe un char y nos devuelve el código ascii de ese char
- q) Como parámetro se recibe una lista de palabras y una palabra. Se devuelve true si la palabra está dentro de la lista de palabras
- r) Crea una enum llamada TipoElemento con valores ARCHIVO y CARPETA. A continuación haz una variable que guarde una lambda que como parámetro reciba la ruta de un archivo. En caso de que no exista, lanza una IllegalArgumentException. Si existe, nos devuelve la constante ARCHIVO o CARPETA correspondiente a esa ruta
- s) Como parámetro recibe dos números y nos devuelve la suma de todos los números que hay entre ellos (ambos inclusive)
- t) Como parámetro recibe dos listas de palabras llamadas “origen” y “destino”. Añade todas las palabras de la lista “origen” a la lista “destino”.

Ejercicio 2: Usa las variables creadas en el ejercicio anterior para ejecutar con ellas las siguientes acciones:

- a) Mostrar en pantalla la frase “hola mundo”
- b) Calcular el cuadrado de 25
- c) Mostrar en pantalla la hora actual
- d) Crear un archivo llamado prueba.txt
- e) Comprobar que la frase “viva el tema 7” tiene 3 espacios
- f) Calcular el mayor de los números 100 y 235
- g) Sacar la letra del dni 11111111
- h) Comprobar que 15 años no es mayor de edad
- i) Obtener la fecha actual en formato día/mes/año
- j) Comprobar que el número 30 no es primo y el 7 si lo es
- k) Hacer una pausa de 5 segundos
- l) Obtener el código ascii de la letra L
- m) Comprobar que “martes” está dentro de la lista de palabras con los días de la semana
- n) Comprobar que c:\windows es una carpeta
- o) Obtener la suma de todos los números comprendidos entre 5 y 28 (ambos incluidos)

2.3.- Simplificación de las expresiones lambda

En muchos casos es posible simplificar la escritura de las expresiones lambda, de forma que escribamos mucho menos código. En general, las expresiones lambda admiten estas simplificaciones:

- Los tipos de datos de los parámetros se puede quitar
- Cuando solo hay un parámetro, y le hemos quitado el tipo de dato, podemos quitar también sus paréntesis
- Cuando la expresión lambda solo contiene una línea con un return, podemos quitar la palabra return y las llaves, poniendo el resultado del return justo después de la flecha.

Vamos a ver cómo podemos simplificar las expresiones lambda que escribimos en los ejemplos del apartado anterior.

- En la expresión lambda que calcula la raíz cuadrada de un número podemos hacer lo siguiente:
 - o Quitamos el tipo de dato del parámetro (esto es algo que se puede hacer siempre, ya que Java coge el tipo del parámetro de la palabra Function)

```
1. Function<Integer,Double> codigo2 = (n) -> {  
2.     return Math.sqrt(n)  
3. };
```

- o Como solo ha quedado un parámetro, le podemos quitar los paréntesis

```
1. Function<Integer,Double> codigo2 = n -> {  
2.     return Math.sqrt(n)  
3. };
```


- o Como la lambda en realidad solo consta de un return, podemos quitar la palabra return, las llaves y poner el resultado que queremos devolver justo después de la flecha ->

```
1. Function<Integer,Double> codigo2 = n -> Math.sqrt(n);
```

Como vemos, la lambda se ha simplificado considerablemente y la lectura del código y de lo que guarda la lambda se ha hecho muy sencilla.

- Aplicamos las reglas anteriores para simplificar la lambda que recibe una frase y nos devuelve true si su número de caracteres es par:

```
1. Predicate<String> codigo3 = frase -> frase.length() % 2 == 0;
```

Ejercicio 3 : Simplifica todas las expresiones lambda escritas en el ejercicio 1

2.4.- Inmutabilidad y funciones puras



La programación funcional se basa en el empleo constante de tipos de datos inmutables y funciones puras. De hecho, en los lenguajes puramente funcionales se exige que todo sea así. Los tipos inmutables ya los conocemos, y son aquellas clases en las que las variables de instancia no cambian, y su valor siempre es el mismo.

En Java una **función pura** es un código fuente (se puede aplicar tanto a métodos como a expresiones lambda) que para unas entradas concretas, siempre producirá el mismo resultado de salida. Además, dicho código no puede tener efectos colaterales, como modificar variables de instancia, escribir en archivos, mostrar datos en pantalla o lanzar excepciones en caso de que puedan fallar.

La verdadera utilidad de usar tipos inmutables y funciones puras no se aprecia del todo en Java, debido a que la mutabilidad es necesaria en este lenguaje. Aún así, emplearlas es buena costumbre para facilitar el mantenimiento y ayudar a la corrección de errores. Como las funciones puras son completamente predecibles, cuando un resultado producido por el programa no es correcto, es fácil encontrar el lugar donde realizar la correspondiente modificación, sin tener en cuenta que su código pueda afectar a otras partes del programa.

Ejercicio 4 : ¿Cuáles de las siguientes clases son inmutables?

- | | | |
|----------------------|-----------------------|------------------|
| a) String | i) Los records | e) Graphics |
| b) LocalDate | j) StringBuilder | f) File |
| c) DepositoAgua | k) Los arrays | g) HashMap |
| d) ArrayList<String> | l) MarcadorBaloncesto | h) BufferedImage |

Ejercicio 5 : Repasa las expresiones lambda del ejercicio 1 y di cuáles son funciones puras

2.3.- Restricciones de las expresiones lambda

Las expresiones lambda que hemos escrito hasta ahora pueden crear sus propias variables, pero ¿qué ocurre con las demás variables que ya existan en el programa? Por ejemplo, ¿puede una expresión lambda consultar o modificar una variable de instancia? ¿y una variable local que ya existiera en el programa?

En el lenguaje Java no hay ningún problema con las variables de instancia, y una lambda puede hacer con ellas lo que quiera. Sin embargo, hay una restricción muy importante que afecta a las variables locales, y es que una lambda solo puede consultar aquellas variables locales que sean **finales** (o sea, constantes) o **efectivamente finales** (no cambian nunca, y por tanto, son como si fuesen “finales” aunque no lleven dicha palabra)

Ejemplo: Si tenemos un contador que no ha cambiado, y queremos mostrarlo en una lambda, podremos hacerlo sin ningún problema:

```
int contador = 0;
Runnable incrementador = () -> {
    System.out.println(contador);
};
```

Esto se debe a que la variable “contador” es efectivamente final, porque su valor nunca cambia en el programa principal.

Ejemplo: Si tenemos una variable cuyo valor ha cambiado, no podremos usarla en la lambda:

```
int contador = 0;
contador = 1;
Runnable incrementador = () -> {
    System.out.println(contador);
};
```

Variable used in lambda expression should be final or effectively final

Ejemplo: Si tenemos un contador y queremos incrementarlo en una lambda, no podremos:

```
int contador = 0;
Runnable incrementador = () -> {
    contador++;
};
```

Variable used in lambda expression should be final or effectively final

Estas restricciones son propias de Java y otros lenguajes no las poseen. Sin embargo, no tienen tanta importancia porque en general, **nunca** se recomienda que las expresiones lambda puedan modificar variables externas a ellas (como las variables locales o las variables de instancia). Recordemos que en programación funcional se trabaja siempre con **funciones puras**, que son aquellas que no tienen efectos colaterales, modificando cosas externas a su código fuente.

3.- Funciones de orden superior

En este apartado vamos a ver la utilidad de las expresiones lambda en Java, y como por si solas, su uso puede simplificar mucho trabajo. Para ello, vamos a poner ejemplos de métodos que se han añadido a las librerías de Java que reciben expresiones lambda como parámetros. Esos métodos activan internamente la lambda recibida como parámetro y consiguen así hacer cosas interesantes, que de otra forma, nos costarían mucho trabajo.

En Java una **función de orden superior (higher order function)** es un método que recibe como parámetro un tipo que almacena código fuente, y al que por tanto, podemos pasar una expresión lambda.

Veamos algunos ejemplos de funciones de orden superior que encontramos en las librerías de Java y cómo nos ayudan mucho, simplificándonos usar bucles.

3.1.- Borrar elementos de una lista

Ejemplo: Vamos a realizar un programa que cree una lista con los días de la semana y nos elimine de esa lista los días que contienen la letra "a"

Comenzaremos haciendo la versión tradicional del programa para luego compararla con la versión funcional. De la manera clásica, hay que crear un List<String> con los días y luego, realizamos un bucle que los recorra y elimine los que tienen la "a". Puesto que los días van a borrarse de la lista, la lista deberá ser mutable.

El bucle que recorrerá la lista va a ser un for que recorreremos hacia atrás, porque la eliminación de un día que contenga la "a", desplazará a la izquierda el resto de días, y si el bucle se recorriera hacia adelante, el desplazamiento que se produciría al borrar un día haría que se pudiera dejar de procesar algún día. Por tanto, el bucle debe hacerse así:

```
1. public class Programa{
2.     public static void main(String[] args){
3.         List<String> lista=new ArrayList(
4.             List.of("lunes","martes","miércoles","jueves","viernes","sábado","domingo")
5.         );
6.         for(int i=lista.size()-1;i>=0;i--){ // recorrido hacia atrás de la lista
7.             String dia=lista.get(i); // saco de la lista el día que estoy recorriendo
8.             if(dia.contains("a")){
9.                 lista.remove(i); // si el día contiene "a", es borrado de la lista
10.            }
11.        }
12.        System.out.println(lista.toString()); // muestro la lista de días en pantalla
13.    }
14. }
```

Vamos ahora a realizarlo de otra forma, usando un método que encontramos en la interfaz **Collection<T>** (que recordemos, es padre de **List<T>**):

<<interface>>
Collection<T>
+ boolean removeIf(Predicate<T> p)

El nombre de este método ya es bastante descriptivo: **removeIf** (borrar si). Este método recorre internamente la lista y activa sobre cada elemento el **Predicate<T>** recibido como parámetro. Aquellos objetos que al aplicarles el Predicate devuelvan true, serán borrados de la lista. De esa forma, se consigue borrar todos los objetos de la lista que cumplen la condición definida mediante una expresión lambda que se pasa como parámetro.

Una vez creada la lista, la solución del ejercicio es tan fácil como pasar a ese método una lambda que devuelva true si un día contiene la letra "a":

```
1. lista.removeIf( dia -> dia.contains("a"));
```

Ejercicio 6 : Realiza las siguientes operaciones:

- a) Sobre la lista de días de la semana, borra los días que tengan más de 5 caracteres
- b) Sobre una lista con los números del 1 al 10, borra todos los que sean impares
- c) Sobre una lista con 10 caracteres, borra los que no correspondan a letras minúsculas
- d) Sobre una lista con 5 fechas, borra aquellas que no sean del año actual
- e) Sobre una lista con 5 depósitos de agua, borra aquellos cuyo porcentaje de ocupación supere el 50%
- f) Sobre la lista de días de la semana escritos en minúscula, borra aquellos que al calcular la media de los códigos ascii de sus letras, salga un valor mayor de 110

3.2.- Procesar todos los elementos de una lista

Ejemplo: Vamos a realizar un programa que muestre cada día de la lista de días de la semana, en una línea diferente.

Comenzamos con la versión tradicional. Creamos una lista con los días de la semana, y la recorremos con un for mejorado (también se puede hacer con el for del lenguaje C).

```
1. List<String> lista= List.of("lunes", "martes", "miércoles", "jueves", "viernes", "sábado", "domingo");
2. for(String dia:lista){
3.     System.out.println(dia);
4. }
```

Para la versión funcional, nos ayudaremos del siguiente método de la interfaz **Iterable<T>** (que recordemos, es padre de **Collection<T>** y por tanto, de **List<T>**)

<<interface>>
Iterable<T>
+ void forEach(Consumer<T> c)

Este método recorre internamente la lista y activa sobre cada elemento el **Consumer** que recibe como parámetro. Por tanto, ese método sirve para aplicar una acción (definida mediante una lambda) sobre todos los elementos de una lista.

Ejemplo: Tenemos la lista de días de la semana y queremos mostrarlos en pantalla.

Bastará pasar a `forEach` un **Consumer** que coja un día y lo muestre en pantalla. El método `forEach` aplicará dicho **Consumer** a todos los elementos de la lista.

```
1. List<String> lista= List.of("lunes", "martes", "miércoles", "jueves", "viernes", "sábado", "domingo");
2. lista.forEach( dia -> System.out.println(dia) );
```

Ejercicio 7 : Realiza las siguientes operaciones:

- Crear una lista con 5 muñecos de Mario (puedes inventar sus posiciones) y hacer que todos ellos salten
- Crear una lista con 5 depósitos de agua vacíos y añadirles a todos un litro
- Crear un archivo llamado **dias.txt** y grabar en él los nombres de los días de la semana, cada uno en una línea diferente del archivo.
- Crear una lista con los 5 objetos **Rectangle** que tú quieras y dibujarlos en la **CapaCanvas** de una **ConsolaDAW**.
- Rellenar una lista con los días de la semana y crear una carpeta en tu ordenador (da igual el lugar) por cada día de la semana
- Rellenar una lista con los días de la semana y crear una carpeta en tu ordenador (da igual el lugar) por cada día de la semana

3.3.- Actualizar todos los elementos de una lista

Si tenemos un **List<T>**, podemos reemplazar cada uno de los objetos por otro diferente (pero del mismo tipo T) usando el siguiente método de la interfaz **List<T>**:

<<interface>>
List<T>
+ void replaceAll(Function<T,T> f)

Este método internamente recorre la lista y activa la **Function** que recibe como parámetro sobre cada elemento de la lista, produciendo un nuevo elemento de tipo T que reemplaza al original. De esa forma, la lambda pasada como parámetro actúa como una transformación que sirve para reemplazar todos los elementos de la lista original por sus transformados por la lambda.

Es de destacar que este método no devuelve nada, sino que actualiza la lista que recibe como parámetro. Por tanto, dicha lista deberá ser **mutable** (por ejemplo, un **ArrayList**), o si no, se lanzará una excepción.

Ejemplo: Tenemos los días de la semana en una lista y queremos pasarlos a mayúsculas

Primero lo haremos de la forma tradicional. Haremos un for del lenguaje C (no sirve el for mejorado, ya que no permite hacer modificaciones en la lista recorrida) y en cada posición, reemplazamos el día de la semana por su versión en mayúsculas.

```
1. List<String> lista=new ArrayList(  
2.     List.of("lunes","martes","miércoles","jueves","viernes","sábado","domingo")  
3. );  
4. for(int i=0;i<lista.size();i++){  
5.     String dia = lista.get(i);  
6.     String diaMayusculas = dia.toUpperCase();  
7.     lista.set(i,dia); // actualiza la posición "i" de la lista  
8. }
```

Para realizar lo mismo de manera funcional, basta con pasar al método `replaceAll` una lambda que coja un día y lo pase a mayúsculas, así:

```
1. List<String> lista= List.of("lunes","martes","miércoles","jueves","viernes","sábado","domingo");  
2. lista.replaceAll( dia -> dia.toUpperCase() );
```

Ejercicio 8 : Realiza las siguientes operaciones:

- Reemplazar cada día de la semana por su versión en inglés.
- Sustituir, en una lista rellena con los números de 0 a 10, cada número por su cuadrado.
- Sustituir, en una lista de fechas (puedes inventártelas), cada fecha por la fecha que se obtiene sustituyendo el año por el año actual
- Sustituir, en una lista de objetos `Rectangle` (inventáelos), cada rectángulo por un objeto `Rectangle` que sea un cuadrado de igual área que el rectángulo sustituido
- Sustituir, en una lista de `String`, cada `String` por él mismo puesto al revés
- Sustituir, en una lista de `DepositoAgualInmutable` (inventáelos), cada depósito por el que resulta de añadirle un litro de agua

3.4.- Ordenar una lista por distintos criterios

Como sabemos, el método **`Collections.sort`** sirve para ordenar de menor a mayor una lista de objetos de tipo **`List<T>`**, siendo `T` una clase que implementa la interfaz **`Comparable<T>`**. Dicha interfaz define un orden en la clase `T`, llamado **orden natural**, que es el usado por `Collections.sort` para ordenar los elementos de la lista de menor a mayor.

El inconveniente de este enfoque es que solo puede haber un criterio de ordenación (el orden natural), que es el usado para implementar la interfaz `Comparable<T>`. Si por ejemplo, tenemos una clase `Persona` y queremos ordenar un `List<Persona>`, solo podríamos hacerlo por un criterio como el nombre o la edad. Implementando `Comparable<T>`, solo podríamos usar como criterio el orden natural que hubiéramos programado en la clase `Persona`, y en las aplicaciones es muy frecuente querer ordenar una lista por varios criterios (nombre, edad, sueldo, apellidos, etc).

La solución al problema consiste en usar la siguiente sobrecarga del método **Collections.sort**

Collections
+ static void sort(List<T> lista, Comparator<T> comparador)

Este método recibe como primer parámetro la lista que queremos ordenar, y como segundo un objeto **Comparator<T>**, que es un objeto que define el criterio por el que se desea comparar. El método no devuelve nada, pero ordena de menor a mayor los objetos de la lista recibida como parámetro.

Aunque es posible implementar “a mano” la interfaz **Comparator<T>**⁵, lo más fácil es usar el método **Comparator.comparing**, que recibe una expresión lambda que transforma un objeto de tipo T en el valor usado para comparar los objetos, y nos devuelve el **Comparator<T>** que necesitamos.

Ejemplo: Queremos ordenar la lista de días de la semana alfabéticamente

Los días de la semana son **String** y dicha clase implementa la interfaz **Comparable** ordenando alfabéticamente. Dicho con otras palabras, el orden natural de los **String** es el orden alfabético. Así que no hay que hacer nada, y la solución consiste en llamar al método **Collections.sort** que ya conocíamos y los días de la semana se ordenarán alfabéticamente.

1. `List<String> dias = List.of("lunes", "martes", "miércoles", "jueves", "viernes", "sábado", "domingo");`
2. `Collections.sort(dias); // ordena los días según el orden natural de la clase String (alfabéticamente)`

Ejemplo: Queremos ordenar la lista de días de la semana de menor a mayor número de caracteres.

En el ejemplo anterior hemos visto que el orden natural de los **String** es la ordenación alfabética, y por tanto usar directamente **Collections.sort()** solo puede ordenar alfabéticamente. Para poder hacer este segundo ejemplo necesitamos pasar al método **Collections.sort** un **Comparator<String>** que compare los **String** por número de caracteres.

La forma más simple de obtenerlo consiste en usar el método **Comparator.comparing** y pasarle una lambda que transforme cada **String** en su número de caracteres, así:

1. `List<String> dias = List.of("lunes", "martes", "miércoles", "jueves", "viernes", "sábado", "domingo");`
2. `Collections.sort(dias, Comparator.comparing(dia -> dia.length()));`

Observar lo sencillo que se hace con la programación funcional ordenar una lista por un criterio cualquiera.

⁵ De hecho, es posible crear un **Comparator<T>** mediante una expresión lambda que reciba dos objetos de tipo T y nos devuelva un **int** con la distancia entre las cualidades que queremos comparar, de forma análoga a como se implementa la interfaz **Comparable<T>**

Ejercicio 9: Programa la siguiente clase Empleado y crea una lista mutable con los empleados descritos en la tabla. A continuación, realiza de manera funcional estas acciones:

Empleado
- String nombre - String departamento - double sueldo
+ Empleado(String nombre, String departamento, double sueldo) + getters y setters para todas las propiedades

Nombre	Departamento	Sueldo
Juan Pérez	Informática	1200
Antonio Castillo	Contabilidad	1500
María López	Informática	1500
Luis Martínez	Informática	2000
Rigoberta Pérez	Contabilidad	4000

- Borrar los empleados que ganen más de 2500 euros
- Subir un 10% el sueldo a todos los empleados
- Crear un archivo llamado empresa.txt en el que en cada línea, esté escrito el nombre de un empleado y su sueldo.

Ejercicio 10: Un profesor utiliza el siguiente record **CorreccionExamen**, para almacenar las preguntas que cada alumno ha tenido bien y mal, tras corregir los exámenes.

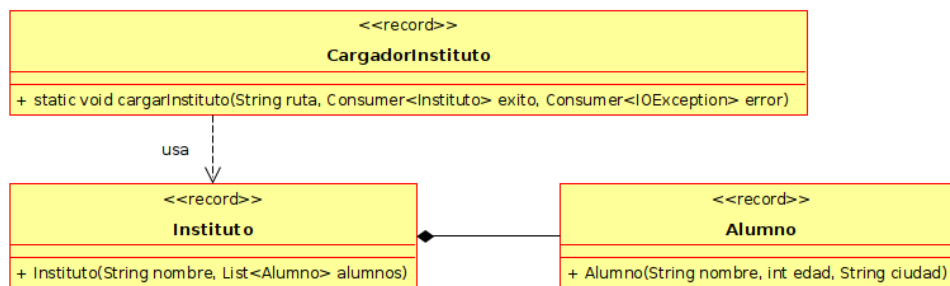
<<record>> CorreccionExamen
+ CorreccionExamen(String alumno, int numeroPreguntas, int respuestasCorrectas) + int respuestasIncorrectas() + double getNota(BiFunction<Integer,Integer,Double> criterioCorreccion)

- Variables de instancia:
 - alumno:** Es el nombre de un alumno
 - numeroPreguntas:** Es el número total de preguntas del examen
 - respuestasCorrectas:** Es el número de preguntas que el alumno ha tenido bien
- Métodos:
 - Constructor:** Comprueba que los números de preguntas y respuestas correctas sean positivos y que el de respuestas correctas no sea mayor que el de preguntas. Si no es así, se lanzará una `IllegalArgumentException` indicando lo que pasa.
 - respuestasIncorrectas:** Devuelve el número de preguntas que el alumno ha tenido mal. Se supone que cada pregunta que no esté bien, está mal.
 - getNota:** Devuelve la nota del examen. Como parámetro recibe una **BiFunction** con la fórmula que se usará para calcular la nota. Dicha **BiFunction** es una lambda que recibe como parámetros el número de preguntas correctas, el de preguntas incorrectas y contiene el código fuente con la fórmula que calcula la nota del examen a partir de dichos datos.

Tienes que programar el record `CorreccionExamen` y después hacer un programa que muestre en pantalla la nota de un alumno que ha tenido bien 8 preguntas en un examen de 12 preguntas, en estos supuestos:

- Si la nota se obtiene dividiendo la cantidad de preguntas bien por la cantidad total de preguntas, y multiplicando todo por 10
- Como antes, pero antes del cálculo cada pregunta mal quita una bien.
- Como antes, pero antes del cálculo cada dos preguntas mal quitan una bien

Ejercicio 11: Programa el siguiente diagrama de clases:



- Alumno:** Es un alumno de un instituto, que tiene un nombre, edad y ciudad.
- Instituto:** Es un instituto, que tiene un nombre y una lista de alumnos
- CargadorInstituto:** Es una clase que posee un método llamado **cargarInstituto**, que abre el archivo cuya ruta se pasa como parámetro y rellena una lista con todos los alumnos del instituto que hay en ese archivo. Se supone que el archivo posee una línea por cada alumno, con el siguiente formato: **nombre;edad;ciudad**

*Pista: Se creará una lista donde se irán guardando los alumnos que se lean del archivo. Una vez que se lean todos los alumnos del archivo, se creará un instituto cuyo nombre será igual al nombre del archivo, y su lista de alumnos será la que se ha formado anteriormente. Cuando el instituto esté creado, se activará el Consumer **éxito** pasándole dicho instituto. Si se produce un error durante la lectura del archivo, se activará el Consumer **error** pasándole la IOException producida.*

Cuando termines el diagrama, haz este programa de pruebas:

Crea un archivo llamado **hlanz.csv** y rellénalo con 10 alumnos con el formato indicado anteriormente. A continuación, carga dicho archivo y en caso de cargarse correctamente, se muestra el nombre del instituto y después el nombre y edad de cada alumno del instituto. Si se produce un error, se muestra un mensaje de error.

☞ **Importante:** En este ejercicio estamos viendo un patrón que se usa mucho en programación funcional. Tradicionalmente los métodos devuelven true/false o un objeto (null en caso de fallo), pero en programación funcional estos métodos se convierten en funciones de orden superior que reciben una lambda que actúa sobre el resultado si todo va bien y otra lambda que se ejecuta si se produce un error.

4.- Streams

La **Stream API** es la librería de Java que permite sacar el máximo partido a la programación funcional en el lenguaje Java. Con su uso, podemos realizar tareas de procesamiento de datos simplemente describiendo lo que queremos que suceda, en lugar de programar una solución explícita mediante bucles y condicionales.



La idea es partir de una **fuentes de datos** (como por ejemplo, una lista de objetos), y obtener a partir de ella un **stream**, que es una especie de “cadena de montaje” por la que van fluyendo sus elementos de uno en uno. Sobre esa “cadena de montaje” ponemos módulos que realizan transformaciones sobre los elementos, como por ejemplo, filtrado, transformación en otras cosas, etc. Al final de la cadena, tenemos el resultado que deseamos. Como podemos imaginar, dichas transformaciones se expresarán por medio de expresiones lambda.



Fig 3 – Una cadena de montaje es un proceso de producción en el que la fabricación de un producto se descompone en una serie de etapas que siguen un orden preestablecido

En Java usaremos el método **stream()** de las listas para iniciar una “cadena de montaje” sobre la que fluyen de uno en uno, todos los elementos de la lista. A partir de ahí, llamaremos de forma encadenada a métodos que van procesando los elementos que viajan por la cadena. Los más básicos son⁶:

- **filter:** Recibe un **Predicate** y elimina de la cadena de montaje los elementos que no lo cumplan (o sea, al activar el Predicate sobre el elemento, nos devuelva false).
- **map:** Recibe un **Function** y lo que hace es transformar cada elemento de la lista en otro elemento (posiblemente de un tipo de dato diferente, pero no pasa nada)
- **mapToInt:** Transforma cada elemento de la lista en un número entero, formándose así una “cadena de montaje” por la que fluyen números enteros.
- **flatMap:** Permite transformar un objeto en una cadena de montaje, que se engancha con la cadena actual. Es un método importantísimo en programación funcional.
- **sort:** Transforma la cadena de montaje en otra en la cual todos los elementos del origen de datos están ordenados de menor a mayor. Como parámetro recibe el Comparator que servirá para hacer la ordenación. Si no se pasa ninguno, se usará el orden natural que haya sido programado en la clase.

⁶ Hay muchos más métodos que podemos aplicar en la “cadena de montaje”, pero nos restringiremos solo a estos dos.

- **distinct:** Filtra los elementos de la cadena de montaje, eliminando duplicados
- **limit:** Permite indicar el tamaño de la cadena de montaje y solo los primeros elementos del origen de datos serán procesados en ella.
- **skip:** Permite saltarse los primeros elementos del origen de datos.

Estos métodos se denominan **operaciones intermedias**, y actúan como “módulos de la cadena de montaje” que transforman los elementos que llegan a ellos, en otros elementos. De esta forma los elementos originales van siendo filtrados y transformados conforme viajan por la cadena de montaje.

Por último, al final de la “cadena de montaje” tenemos una **operación terminal**, que es un método que recibe los objetos (filtrados y transformados por las operaciones intermedias) y los procesa, haciendo algo con ellos. Los más importantes son:

- **count:** Cuenta cuántos objetos han llegado al final de la cadena de montaje
- **forEach:** Recibe un **Consumer** y lo aplica sobre cada elemento que llega al final de la cadena de montaje
- **toList:** Guarda en una lista todos los objetos que llegan al final de la cadena
- **collect:** Permite fusionar todos los objetos que llegan al final de la cadena de montaje en uno, que sirve de resultado del proceso. Como parámetro recibe un objeto “coleccionar” objetos procedentes de la cadena de montaje. La clase **Collectors** posee muchos de estos objetos predefinidos, y el que usaremos con más frecuencia será **Collectors.joining**, que forma un String con todos los objetos que llegan al final de la cadena de montaje. Dicho String se forma llamando al método **toString** de cada objeto y es posible indicar el carácter usado para separar unos de otros

Cuando la cadena es de números, los métodos **min** y **max** nos permiten obtener el número más pequeño o más grande, sin necesidad de pasarles un Comparator.

Vamos a ver ejemplos de todo lo que hemos descrito.

Ejemplos: Para realizar los ejemplos, vamos a partir del siguiente record Alumno y una lista de alumnos con los que tenemos que hacer determinadas operaciones que normalmente nos llevarían a usar bucles y condicionales.

<<record>>	
Alumno	
+ Alumno(String nombre, int edad, String ciudad, List<String> asignaturas)	

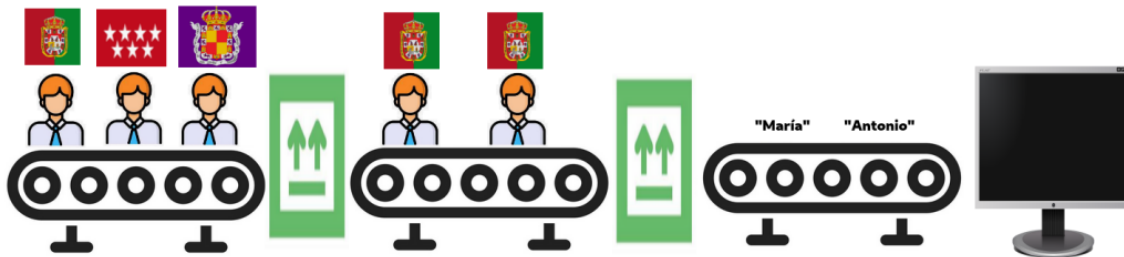
```

1. public static void main(String[] args) {
2.     List<Alumno> alumnos = List.of(
3.         new Alumno("Antonio", "Granada", 20, List.of("Programación", "Bases de datos")),
4.         new Alumno("Luis", "Jaén", 35, List.of("Entornos", "Bases de datos")),
5.         new Alumno("María", "Granada", 22, List.of("Programación", "Entornos", "Sistemas")),
6.         new Alumno("Jose", "Almería", 50, List.of("Programación")),
7.         new Alumno("Arturo", "Jaén", 19, List.of("Programación", "Entornos", "Sistemas"))
8.     );
9. }
```

- Mostrar en pantalla el nombre de los alumnos que son de Granada

Cogemos la lista y con el método **stream()** formamos una cadena de montaje por la que van pasando todos los objetos Alumno de la lista. Entonces usamos el método **filter**, que va a actuar como un filtro que solo deja pasar los alumnos de Granada. Dicho método recibe un Predicate que cogerá un objeto Alumno y devolverá true cuando sea de Granada. De esa forma, solo los alumnos que sean de Granada pasarán el filtro. A continuación, cogemos cada objeto Alumno y con **map** le aplicamos una lambda que lo transforma en su nombre, pasando así a tener una cadena de montaje de String. Por último, ponemos un **forEach** que coja cada uno de esos String que llegan al final de la cadena, y se muestre en pantalla

```
1. alumnos.stream()      // cadena de montaje con todos los alumnos
2.   .filter(alumno -> alumno.nombre().equals("Granada")) // solo deja pasar alumnos de Granada
3.   .map(alumno -> alumno.nombre()) // transforma cada objeto Alumno en un String con su nombre
4.   .forEach(nombre -> System.out.println(nombre)); // muestra cada String en pantalla
```



También se puede hacer de otra forma, filtrando primero los alumnos y haciendo que el **forEach** muestre el nombre del objeto Alumno que llega al final de la cadena, así:

```
1. alumnos.stream()      // cadena de montaje con todos los alumnos
2.   .filter(alumno -> alumno.nombre().equals("Granada")) // solo deja pasar alumnos de Granada
3.   .forEach(alumno -> System.out.println(alumno.nombre())); // muestra el nombre del alumno
```

Aunque este segundo enfoque es más corto, en realidad es peor por dos motivos:

1. Se prefiere siempre que las expresiones lambdas sean muy cortas. Es mejor usar lambdas pequeñas que hagan poca cosa. De hecho, al final del tema veremos que ese tipo de lambdas se pueden simplificar aún más usando **referencias a métodos**.
2. El código escrito de esa forma es más legible, ya que se ven con mayor claridad todos los pasos del proceso (es mejor tener muchos pasos pequeños a uno muy grande)
3. A la hora de depurar, usar ese tipo de lambdas ayuda a ver con mayor claridad los objetos que en cada cadena de la cadena de montaje

Para depurar una expresión como la anterior, se usa la operación intermedia **peek**, que lo recibe como parámetro un **Consumer** con una acción a realizar sobre el objeto, continuando después por la cadena de montaje. Típicamente se usa peek para mostrar el objeto por pantalla y así saber lo que está sucediendo en la cadena.

Por ejemplo, si queremos ver los alumnos justo tras aplicar el filtro, escribiríamos esto:

```

5. alumnos.stream()      // cadena de montaje con todos los alumnos
6.   .filter(alumno -> alumno.nombre().equals("Granada")) // solo deja pasar alumnos de Granada
7.   .peek(alumno -> System.out.println(alumno.toString())); // vemos los alumnos que pasan el filtro
8.   .map(alumno -> alumno.nombre()) // transforma cada objeto Alumno en un String con su nombre
9.   .forEach(nombre -> System.out.println(nombre)); // muestra cada String en pantalla

```

- Mostrar en pantalla el nombre de los alumnos mayores de edad

Es muy similar al anterior. Con **stream** iniciamos el stream por el que pasan todos los alumnos y con **filter**, solo dejamos pasar los mayores de edad. En este caso, el Predicate que le pasamos devolverá true cuando el alumno sea mayor de edad. El ejemplo termina transformando con **map** cada objeto Alumno en su nombre, y con **forEach** mostrándolos.

```

1. alumnos.stream()      // cadena de montaje con todos los alumnos
2.   .filter(alumno -> alumno.edad()>=18) // solo deja pasar alumnos mayores de edad
3.   .map(alumno -> alumno.nombre()) // transforma cada objeto Alumno en un String con su nombre
4.   .forEach(nombre -> System.out.println(nombre)); // muestra cada String en pantalla

```

- Mostrar en pantalla el nombre de los alumnos de Granada menores de edad, ordenados alfabéticamente

En este caso, comenzamos con **stream** la cadena de montaje por la que pasan todos los alumnos y ahora pondremos dos filtros. El primero dejará pasar solo los alumnos de Granada, de forma que pasemos a una cadena formada solo por alumnos de Granada. Sobre esos, ponemos un segundo filtro que solo dejará pasar alumnos mayores de edad. Tras la aplicación de los dos filtros, la cadena de montaje solo tiene alumnos de Granada mayores de edad. A continuación, transformamos con **map** cada alumno en su nombre y así pasamos a una cadena de montaje formada solo por nombres de alumnos. Por último, con **forEach** mostramos en pantalla cada nombre que llega al final de la cadena.

```

1. alumnos.stream()      // cadena de montaje con todos los alumnos
2.   .filter(alumno -> alumno.ciudad().equals("Granada")) // solo deja pasar alumnos de Granada
3.   .filter(alumno -> alumno.edad()>=18) // ahora retira los alumnos menores de edad
4.   .map(alumno -> alumno.nombre()) // transforma cada objeto Alumno en un String con su nombre
5.   .forEach(nombre -> System.out.println(nombre)); // muestra en pantalla los nombres que llegan

```

- Mostrar en pantalla el número de alumnos mayores de edad de Granada

Comenzamos como antes, y tras quedarnos con los alumnos de Granada mayores de edad, usamos **mapToInt** para transformar cada objeto Alumno en su edad, que es un número entero. Con esto pasamos a una cadena de montaje que solo tiene números de tipo básico **int**, y dicha cadena tiene el método **max** que nos da el máximo

- Obtener un String con el nombre de todas las ciudades de los alumnos separados por comas y mostrarlo en pantalla

En este caso comenzamos creando con **stream** la cadena de montaje para procesar los objetos Alumnos de la lista, y con **map** transformamos cada objeto Alumno en su nombre. Usando **distinct** nos aseguramos de que todos los nombres que viajan por la cadena sean diferentes. Por último, el método **collect** nos permite juntar todos esos String en un solo

objeto. Como “coleccionador” usaremos **Collectors.joining**, que nos da un objeto que sabe juntar todos los objetos que llegan en un String (usamos la coma como separador).

```
1. String ciudades = alumnos.stream() // cadena de montaje con todos los objetos Alumno
2.   .map(alumno -> alumno.ciudad()) // transformamos cada objeto Alumno en un String con su ciudad
3.   .distinct() // eliminamos las ciudades repetidas
4.   .collect(Collectors.joining(",")); // juntamos en un String todas las ciudades recibidas
```

- Guardar en un List<String> los nombres de los alumnos que empiezan por A

La forma de resolver el ejemplo consiste en obtener el stream por el que pasen objetos Alumno, transformarlos en su nombre y filtrarlos para que solo pasen los nombres que empiecen por la A (también se puede hacer al revés, filtrando primero los alumnos y luego transformándolos en sus nombres). Cuando ya tenemos los nombres, el método **toList** nos los recoge en una lista (también se puede usar **collect(Collectors.toList())**).

```
1. List<String> nombres = alumnos.stream()
2.   .map(alumno -> alumno.nombre())
3.   .filter(nombre -> nombre.startsWith("A"))
4.   .toList();
```

- Mostrar en pantalla las tres menores edades de los alumnos

Podemos hacer este ejemplo de dos formas. La más inmediata consiste en formar un stream por el que pasen los alumnos de la lista y los ordenamos por edad, llamando al método **sorted** con un Comparator que transforma cada alumno en su edad. Hecho eso, lo limitamos a tres alumnos, los convertimos con **map** en su edad, y por último, la mostramos en pantalla.

```
1. alumnos.stream() // stream con todos los objetos Alumno
2.   .sorted(Comparator.comparing( alumno -> alumno.edad() ) ) // ordenamos por la edad
3.   .limit(3) // nos quedamos solo con los tres primeros alumnos
4.   .map( alumno -> alumno.edad() ) // transformamos cada alumno en su edad
5.   .forEach(edad -> System.out.println(edad)); // mostramos cada número en la pantalla
```

Otra forma consiste en obtener el stream por el que pasan los objetos Alumno de la lista y usar **mapToInt** para transformar cada objeto Alumno en su edad. Así pasamos a un stream por el que fluyen números enteros. Con **sorted** ordenamos dicho stream para que los números fluyan de menor a mayor y con **limit** nos quedamos solo con los primeros tres números. Dichos números serán mostrados en pantalla en un **forEach**

```
1. alumnos.stream() // cadena de montaje con todos los objetos Alumno
2.   .mapToInt(alumno -> alumno.edad()) // transformo cada objeto Alumno en un int con su edad
3.   .sorted() // hace que los números del stream (las edades) fluyan ordenados de menor a mayor
4.   .limit(3) // nos quedamos solo con los primeros 3 números
5.   .forEach(numero -> System.out.println(numero)); // mostramos cada número en la pantalla
```

- Mostrar en la pantalla el nombre de todas las asignaturas, sin que se repitan.

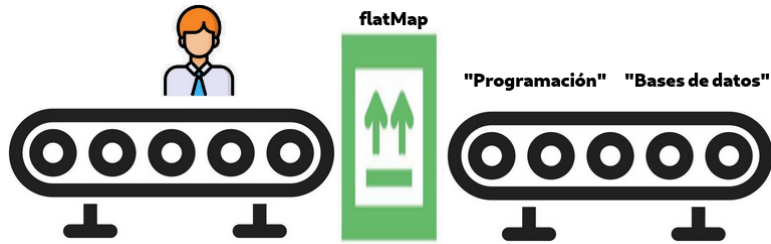
Una primera idea consistiría en transformar cada alumno en su lista de asignaturas y mostrarla en pantalla, pero esto nos daría un problema, y es que las asignaturas se repetirían, y no podríamos hacer nada por solucionarlo, ya que el procesamiento de cada lista de asignaturas es independiente de los demás y no podríamos filtrar asignaturas repetidas.

```

1. alumnos.stream()
2.   .map( alumno -> alumno.asignaturas())
3.   .forEach(
4.       lista -> lista.forEach(
5.           asignatura -> System.out.println(asignatura)
6.       )
7.   );

```

La solución consiste en transformar cada alumno, no en una lista de asignaturas, sino en un **stream** con sus asignaturas, que se “engancha” con el stream en curso. Es como si un objeto Alumno se descompusiera en sus asignaturas y estas empezaran a fluir por la cadena de montaje. Esta operación se denomina **flatMap**.



```

1. alumnos.stream()    // stream con objetos Alumno
2.   .flatMap( alumno -> alumno.asignaturas.stream()) // stream con String de asignaturas
3.   .distinct()       // nos quedamos con asignaturas distintas
4.   .forEach( asignatura -> System.out.println(asignatura)); // las mostramos

```

Una vez que el stream de objetos Alumno se ha transformado en un stream de String con los nombres de asignaturas, ya podemos aplicar los métodos habituales, como **distinct** o **forEach**

4.1.- Optional

El **Optional<T>** es un tipo muy importante en programación funcional. Es un objeto que puede estar vacío, o contener un objeto de tipo T. Aunque no vamos a entrar mucho en detalles, esta clase es muy importante porque tiene métodos que permiten realizar transformaciones (con lambdas) sobre el objeto que hay en el Optional, conservando el Optional vacío en caso de actuar sobre un Optional vacío. Además, su uso evita el temido **NullPointerException**, ya que la “ausencia de información” que tradicionalmente se ha representado con el valor **null**, ahora se representa mediante un Optional vacío, evitando así ese error.



Los stream tienen métodos que devuelven Optional, como por ejemplo:

- **findFirst**: Devuelve un Optional con el primer objeto que llega al final del stream (dicho Optional estará vacío si no llega ningún elemento al final del stream)
- **findAny**: Devuelve un Optional con un objeto cualquiera de los que haya llegado al final del stream
- **min**: Recibe un Comparator y nos devuelve un Optional con el elemento más pequeño de la cadena, según el criterio de ordenación definido en el Comparator. El Optional devuelto estará vacío si no hay objetos para calcularles el mínimo.
- **max**: Recibe un Comparator y nos devuelve un Optional con el mayor elemento de la cadena, de forma análoga al método min.

Veamos ejemplos de estos métodos:

- Hacer un programa que pregunte por teclado el nombre de una ciudad y nos muestre en pantalla el nombre de un alumno cualquiera de esa ciudad, o el mensaje “no hay alumnos en esa ciudad”

Tras preguntar por teclado la ciudad, comenzamos obteniendo un stream por el que fluyen todos los alumnos. A continuación, ponemos un filtro para que solo pasen objetos Alumno cuya ciudad sea la deseada, y después, transformamos cada objeto Alumno en su nombre. Por último, usando **findAny** obtenemos uno de esos nombres (normalmente es el primero, pero en aplicaciones multitarea puede ser cualquiera). Puesto que el stream puede ser vacío (por ejemplo, si no hay alumnos en esa ciudad), **findAny** devuelve un **Optional<String>**. Para mostrar por pantalla el resultado, el **Optional** posee el método **ifPresentOrElse**, que recibe dos lambdas. La primera es un **Consumer** que procesa el objeto que hay en el optional (en caso de que el **Optional** no esté vacío) y la segunda es un **Runnable** que se ejecuta si el **Optional** está vacío.

```
1. System.out.println("Escribe una ciudad");
2. String ciudad = new Scanner(System.in).nextLine();
3. alumnos.stream() // stream con todos los objetos Alumno
4.     .filter(alumno -> alumno.ciudad().equals(ciudad)) // nos quedamos con alumnos de la ciudad deseada
5.     .map(alumno -> alumno.nombre()) // transformamos cada objeto Alumno en un String con su nombre
6.     .findAny() // devuelve un Optional<String> con cualquiera de los nombres encontrados
7.     .ifPresentOrElse( // según el Optional esté vacío o no, ejecuta una de las siguientes lambdas
8.         nombre -> System.out.println("Alumno encontrado: "+nombre), // muestra el nombre encontrado
9.         () -> System.out.println("No hay alumnos en "+ciudad) // muestra un mensaje de error
10.    );
```

- Hacer un programa que pregunte por teclado el nombre de una ciudad y nos muestre en pantalla la edad más grande que encontramos entre los alumnos mayores de edad de esa ciudad

En este caso comenzamos como en el ejemplo anterior, y con los filtros habituales hacemos que en el stream solo viajen los alumnos mayores de edad de esa ciudad. En ese momento, usando **mapToInt** transformamos cada objeto Alumno en su edad, que es un número entero. Con esto pasamos a un stream que solo tiene números de tipo básico **int**, y dicho stream tiene el método **max** que nos da su valor máximo en forma de **Optional** (si no hay alumnos de esa ciudad, se obtendrá un **Optional** vacío). Usando el método **ifPresentOrElse** como antes, podremos mostrar por pantalla el resultado, o el mensaje de error.

```
1. System.out.println("Escribe una ciudad");
2. String ciudad = new Scanner(System.in).nextLine();
3. alumnos.stream() // stream con todos los objetos Alumno
4.     .filter(alumno -> alumno.ciudad().equals(ciudad)) // nos quedamos con los alumnos de la ciudad
5.     .map(alumno -> alumno.edad() >= 18) // nos quedamos con alumnos mayores de edad
6.     .mapToInt( alumno -> alumno.edad() ) // transformamos cada alumno en su edad
7.     .max() // obtenemos un optional con el valor máximo de las edades
8.     .ifPresentOrElse( // ejecuta una lambda u otra, según el optional encontrado tenga algo, o no
9.         edad -> System.out.println("La edad máxima es: "+edad),
10.         () -> System.out.println("No hay alumnos en "+ciudad)
11.    );
```

- Muestra en pantalla el nombre del alumno con más asignaturas

Comenzamos obteniendo un stream con todos los objetos Alumno y usaremos el método **max** para ordenar los alumnos por número de asignaturas y obtener el máximo. Para ello le pasaremos a dicho método un Comparator que transforme cada alumno en su número de asignaturas.

```
1. alumnos.stream()
2.   .max( Comparator.comparing(alumno -> alumno.asignaturas().size()))
3.   .ifPresentOrElse(
4.       alumno -> System.out.println("El alumno con más asignaturas es: "+alumno.nombre()),
5.       () -> System.out.println("La lista de alumnos está vacía")
6.   );
```

- Hacer un programa que encuentre un alumno de Granada y nos muestre su edad

En este caso comenzamos filtrando los alumnos de Granada y con **findAny** obtenemos un `Optional<Alumno>` que podrá guardar cualquier alumno de Granada, cumpliendo así la primera parte del ejemplo. Una vez que tenemos el `Optional<Alumno>`, podemos usar su método **map** para transformar el Alumno que hay en el Optional en su edad. Obtendremos así un `Optional<Integer>` que contendrá la edad del Alumno (o estará vacío si el `Optional<Alumno>` estaba vacío).

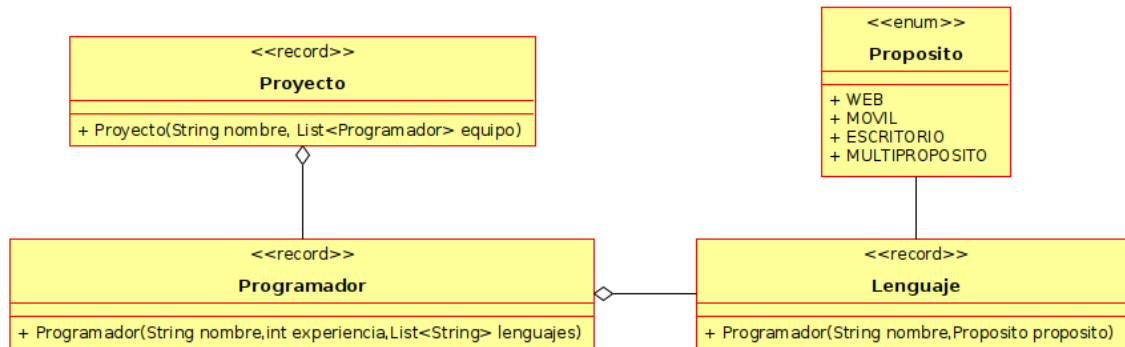
```
1. alumnos.stream()// stream de objetos Alumno
2.   .filter(alumno -> alumno.ciudad().equals("Granada")) // solo alumnos de Granada
3.   .findAny() // encontramos cualquiera de ellos, pasando a un Optional<Alumno>
4.   .map( alumno -> alumno.edad()) // transformamos el alumno en su edad, pasando a Optional<Integer>
5.   .ifPresentOrElse(
6.       edad -> System.out.println("La edad buscada es: "+edad),
7.       () -> System.out.println("No hay ningún alumno en Granada")
8.   );
```

- Encontrar un alumno cualquiera de Granada y mostrar el nombre de su asignatura de menor longitud

Para resolver este ejemplo, primero comenzaremos repitiendo los pasos del ejemplo anterior, filtrando los alumnos para quedarnos con los de Granada, y después obteniendo un alumno cualquiera de ellos. Obtendremos así un `Optional<Alumno>` que contendrá un alumno cualquiera de Granada. Para obtener su asignatura de menor longitud, podríamos usar **map** que transforme el alumno encontrado en la asignatura buscada, pero si se hace, veríamos que nos saldría un `Optional` que contiene otro `Optional`, y se nos complicaría la cosa. Por ese motivo, no vamos a usar **map**, y va a ser más fácil usar el método **flatMap** que tiene la clase `Optional`. Dicho método permite transformar el objeto contenido en el `Optional` en otro `Optional`, que se “engancha” con el que tenemos.

```
1. alumnos.stream()
2.   .filter( alumno -> alumno.ciudad().equals("Granada"))
3.   .findAny()
4.   .flatMap( alumno -> alumno
5.       .asignaturas()
6.       .stream()
7.       .min(Comparator.comparing( asignatura -> asignatura.length()))
8.   ).ifPresentOrElse(
9.       asignatura -> System.out.println("La asignatura buscada es: "+asignatura),
10.      () -> System.out.println("No hay alumnos en Granada")
11.  );
```

Ejercicio 12: Programa el siguiente diagrama de clases, rellena dos listas llamadas **proyectos** y **programadores** con los datos proporcionados, y realiza las operaciones indicadas usando la Stream API



- Programadores (nombre, experiencia, lenguajes):

Luis González	5	C++ (multipropósito), PHP (web)
Antonio Pérez	2	Java (multipropósito), Python (multipropósito)
Juana López	4	Visual Basic (escritorio), Swift (móvil)
Josefa Smith	3	ASP(web), Java (multipropósito), Dart (móvil)

- Proyectos (nombre, programadores):

Banco Granada	Luis González, Juana López, Josefa Smith
MercaRafa	Antonio Pérez, Luis González, Josefa Smith
JorgePhone	Antonio Pérez, Juana López

- Mostrar en pantalla todos los programadores que tienen más de 3 años de experiencia
- Mostrar en pantalla el sueldo de todos los programadores, suponiendo que cada sueldo se obtiene multiplicando por 900 € sus años de experiencia y sumando 100 € por cada lenguaje de programación conocido
- Muestra el nombre del programador de menor experiencia que trabaje en el proyecto MercaRafa
- Mostrar todos los lenguajes que conoce el programador con menos años de experiencia
- Muestra el nombre de todos los proyectos donde trabaja Luis González
- Muestra el nombre y experiencia de todos los programadores, ordenados de menor a mayor experiencia
- Muestra la media del número de lenguajes que conocen los programadores
- Muestra en pantalla el nombre de cada proyecto junto con el nombre de su programador principal (que es el primer programador que aparece en su lista de programadores).
- Muestra el nombre del proyecto con menor número de programadores
- Mostrar en pantalla todos los lenguajes de programación de tipo multipropósito
- Muestra en pantalla la media de experiencia de los programadores del proyecto Banco Granada
- Mostrar en pantalla el nombre de todos los programadores que intervienen en el proyecto Banco Granada, ordenados alfabéticamente.

- m) Mostrar en pantalla el nombre de los lenguajes de programación usados en el proyecto MercaRafa
- n) Muestra en pantalla el nombre de todos los lenguajes multipropósito
- o) Muestra en pantalla el nombre y experiencia de cualquier programador que sepa programar en Java
- p) Muestra el nombre de los proyectos donde la suma de los sueldos de los trabajadores que intervienen supera los 5000 € (el sueldo se calcula como en el apartado b)
- q) Muestra en pantalla el nombre de cualquier proyecto en el que intervenga un programador de que sepa programar un lenguaje para web
- r) Muestra el nombre de los programadores que participan en al menos dos proyectos
- s) Muestra el nombre de cada proyecto, junto con todos los lenguajes de programación que intervienen en él, así: *(desafío: hazlo solo usando programación funcional)*

```
Banco Granada > C++,PHP,Visual Basic,Swift,ASP,Java,Dart
MercaRafa > Java,Python,C++,PHP,ASP,Java,Dart
JorgePhone > Java,Python,Visual Basic,Swift
```

4.2.- Distintos orígenes de datos

En todos los ejemplos y ejercicios que hemos hecho, el origen de los datos del stream ha sido siempre una lista (también se admitirían set y arrays). Aunque eso es una situación muy frecuente, también hay veces en las que querremos formar un stream con datos generados automáticamente. Aquí tenemos algunos métodos para conseguirlo:

- Streams de números enteros: Los métodos **IntStream.range** y **IntStream.rangeClosed** nos permiten formar un stream en el que fluyen todos los números comprendidos entre dos valores A y B que reciben como parámetros (B se excluye en el primer método, y se incluye en el segundo). Los datos que fluyen por el stream son **int**
- Stream infinito: Tenemos dos métodos para conseguirlo
 - o El método **Stream.generate** recibe un **Supplier<T>** que actúa como generador de objetos de tipo T. Se forma un stream infinito llamando al Supplier infinitas veces. Como se puede suponer, es necesario usar **limit** para indicar cuántos elementos queremos generar.
 - o El método **Stream.iterate** recibe un objeto llamado **semilla** y un Function que transforma la semilla en otro objeto, que pasará a ser la nueva semilla. Se forma un Stream infinito con todas las semillas, que habrá que limitar con **limit**

Ejemplo: Vamos a calcular la suma de todos los números entre 50 y 10000 (ambos incluidos).

Como podemos suponer, es impráctico crear una lista “a mano” con todos los números que queremos sumar. Una opción podría ser hacer un bucle que genere esos números y los añada a la lista, y luego usar la Stream API para sumarlos, pero hacer eso sería un poco tonto porque durante el bucle, ya podríamos incrementar un contador que nos diera la suma.

Lo que haremos será usar un stream de números enteros, sobre el que irán fluyendo los números que queremos sumar. Ese enfoque es eficiente, porque durante el procesamiento solo está en memoria el número que fluye por el stream.

```
1. int suma = IntStream.rangeClosed(50,10000).sum();
```

Ejemplo: Queremos calcular la suma de todos los números comprendidos entre 50 y 10000, pero sin incluir los extremos.

En este caso, usamos **IntStream.range**, que excluye el 10000, y con **skip** nos saltamos el primer número del stream, que el 50. De esa forma, se suman todos los intermedios.

```
1. int suma = IntStream.range(50,10000) // stream con todos los números entre 50 y 9999
2.     .skip(1) // se salta el primero (el 50)
3.     .sum(); // los suma todos
```

Ejemplo: Queremos obtener una lista con 100 depósitos de agua vacíos, de forma que el primero tenga 50 litros y el siguiente tenga 20 litros más que el anterior.

La idea es crear un stream con las capacidades (que son números enteros) y transformar cada capacidad en el depósito correspondiente. Para crear el stream con las capacidades, usamos el método **Stream.generate** pasando como semilla el primer valor (50) y como función, una lambda que coge una capacidad y le suma 20. Una vez obtenido el stream con las capacidades, lo limitamos a 100 números, transformamos cada número en el depósito correspondiente y los guardamos todos en una lista con el método **toList**

```
1. List<DepositoAgua> lista = Stream.iterate( 50, capacidad -> capacidad+20)
2.     .limit(100)
3.     .map( capacidad -> new DepositoAgua(0, capacidad))
4.     .toList();
```

Ejemplo: Queremos obtener un List<Integer> con 20 números aleatorios entre 0 y 10

Usaremos el método **Stream.generate** y le pasaremos como generador un Supplier que simplemente genera un número aleatorio entre 0 y 10. De esa forma obtenemos un stream infinito formado por números aleatorios comprendidos entre 0 y 10. Pondremos un limitador a 20 elementos y ya tendremos los números buscados, que guardaremos en una lista con el método **toList**

```
1. List<Integer> lista = Stream.generate( () -> new Random().nextInt(0,10) )
2.     .limit(20)
3.     .toList();
```

Ejercicio 13 : Realiza estas acciones, proporcionando dos formas distintas de hacer cada una:

- Mostrar en pantalla 100 veces la palabra “hola”
- Mostrar en pantalla los 1000 primeros números pares
- Mostrar en pantalla todos los días del mes actual
- Obtener un List<LocalTime> con el tiempo de los siguientes 10 segundos.
- Mostrar en pantalla todas las líneas de un archivo llamado texto.txt

5.- Referencias a métodos

Por último, vamos a ver cómo en algunos casos las expresiones lambda que hemos escrito se pueden sustituir por expresiones más sencillas llamadas **referencias a métodos**.

Una **referencia a un método** es una notación especial con la que podemos reemplazar las lambdas que solo consisten en llamar a un método sobre el parámetro de la lambda, sin hacer nada más.

La referencia al método consiste en sustituir el último punto de la llamada al método por ::

Veamos ejemplos de lambdas que se pueden sustituir por una referencia a un método:

- Mostrar en pantalla el nombre de los días de la semana

La expresión lambda que estudiamos en su momento fue esta:

```
1. List<String> lista= List.of("lunes", "martes", "miércoles", "jueves", "viernes", "sábado", "domingo");
2. lista.forEach( dia -> System.out.println(dia) );
```

Como vemos, la lambda solo consiste en llamar al método `System.out.println` sobre el parámetro de la lambda. Podemos sustituirla por la referencia **`System.out::println`**, así:

```
1. List<String> lista= List.of("lunes", "martes", "miércoles", "jueves", "viernes", "sábado", "domingo");
2. lista.forEach( System.out::println );
```

- Reemplazar cada día de la semana por su versión en mayúscula

En su momento, vimos que eso se hacía así:

```
1. List<String> lista= List.of("lunes", "martes", "miércoles", "jueves", "viernes", "sábado", "domingo");
2. lista.replaceAll( dia -> dia.toUpperCase() );
```

Como vemos, la lambda es únicamente la llamada a un método, sin hacer nada más. Podemos quitarla y reemplazarla por la referencia **`dia::toUpperCase`**

```
1. List<String> lista= List.of("lunes", "martes", "miércoles", "jueves", "viernes", "sábado", "domingo");
2. lista.replaceAll( dia::toUpperCase );
```

- Tenemos una lista de números y queremos mostrar en pantalla la raíz cuadrada de esos números.

Crearíamos una lista con los números, los transformamos en su raíz y los mostramos así:

```
1. List.of(10,50,80).stream()
2.     .map( numero -> Math.sqrt(numero))
3.     .forEach( System.out::println );
```

Como vemos, la lambda del map simplemente consiste en llamar a `Math.sqrt` con el parámetro de la lambda. Eso significa que la podemos reemplazar por **`Math::sqrt`**

```

1. List.of(10,50,80).stream()
2.   .map( Math::sqrt )
3.   .forEach( raiz -> System.out.println(raiz));

```

- Queremos crear un List<Caja> que guarde los días de la semana en cajas.

Creamos la lista de días de la semana, transformamos cada uno en un objeto Caja que lo contiene, y los almacenamos todos en una lista, así:

```

1. List.of("lunes","martes","miércoles","jueves","viernes","sábado","domingo").stream()
2.   .map( palabra -> new Caja(palabra))
3.   .toList();

```

La lambda del map solo consiste en una llamada a un método, que en este caso, es un método constructor. Esa situación se puede sustituir por **Caja::new**, que es la referencia al método constructor de la clase Caja cuando actúa sobre el parámetro de la lambda.

```

4. List.of("lunes","martes","miércoles","jueves","viernes","sábado","domingo").stream()
5.   .map( Caja::new )
6.   .toList();

```

- Tenemos la lista de días de la semana y queremos mostrar en pantalla solo las que tienen longitud par.

La solución sería esta:

```

1. List.of("lunes","martes","miércoles","jueves","viernes","sábado","domingo").stream()
2.   .filter(dia -> dia.length() % 2 == 0)
3.   .forEach( System.out::println );

```

En este caso, la lambda del filter no consiste únicamente en llamar a un método, puesto que además de la llamada a **length** también se calcula el resto de la división entre dos y luego se hace una comparación. Así que esa lambda no se puede sustituir por una referencia a un método.

Ejercicio 14: Repasa todas las lambdas del ejercicio 1 y ejercicio 12 y sustituye por referencias a métodos todas las que sea posible.